



nextwork.org

Query Data with DynamoDB

CH

Arpita Chowdhury

```
$ aws dynamodb get-item --table-name ContentCatalog --key '{"Id":{"N":"101"}}' --consistent-read --projection-expression "Title, ContentType, Services" --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 1.0
  }
}
~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"202"}}' \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "Item": {
    "ContentType": {
      "S": "Video"
    },
    "Title": {
      "S": "Don't miss out!"
    }
  },
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 0.5
  }
}
~ $
```



Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database service that provides fast, predictable performance and automatically scales to handle large amounts of data and traffic. It is useful because it removes the need to manage servers, storage, replication, and backups, allowing developers to focus on their applications rather than infrastructure. DynamoDB's flexible, schema-less design makes it easy to store different types of items without rigid table structures, while its distributed architecture enables single-digit millisecond latency at any scale. It also integrates seamlessly with serverless services like AWS Lambda and offers features such as automatic scaling, built-in security, and high availability, making it ideal for modern, scalable applications.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to create table with AWS CLI, add data to table, run query with console and AWS CLI and set up transaction.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is to face problem deleting resources using AWS CLI so I had to do it manually.



Arpita Chowdhury

NextWork Student

nextwork.org

This project took me...

This project took me 2 hours to complete.



Querying DynamoDB Tables

A partition key is the primary attribute that DynamoDB hashes to decide the physical partition where an item will be placed, ensuring fast and scalable data access.

A sort key is the second part of a composite primary key that allows multiple items with the same partition key to be stored together and sorted in ascending order.

▼ Scan or query items

☐ Scan ☒ Query

Select a table or index: Table - Comment ▼

Select attribute projection: All attributes ▼

Partition key

Attribute: Id Value: I have a question/Just Complete Project #7 Dependencies and CodeA

Sort key - optional

Attribute: CommentDateTime Value: Greater than ▼ 2024-09-01

☐ Sort descending

► Filters - optional

Run Reset



Limits of Using DynamoDB

I ran into an error when I queried for the data because the partition key field was left empty. In DynamoDB, a Query must always specify the partition key, so the console shows an error when it's missing.

Insights we could extract from our Comment table include identifying who posted each comment, how many comments belong to a specific post, and when those comments were made, since the partition key groups comments by post and the sort key orders them by time. These structures make it easy to view comment activity, analyze engagement for a single post, and understand user behavior within that post. However, insights we can't easily extract include system-wide analytics such as the most active commenter overall, the most commented-on posts across the entire application, or any trend that requires sorting, aggregating, or comparing data across multiple partitions. Because DynamoDB does not support joins or cross-partition sorting, and is not optimized for heavy full-table scans, deeper global insights require additional processing outside the table.

Partition key

Attribute

Id

Value

Enter attribute value

 The partition key filter cannot be empty.

Sort key - optional

Attribute

CommentDateTime

Value

Equal to

Enter attribute value

☐ Sort descending

▼ Filters - optional

Attribute name	Condition	Type	Value	
Q PostedBy X	Equal to	String	User Abdulrahman	Remove

Add filter



Running Queries with CLI

A query I ran in CloudShell was an `aws dynamodb get-item` command on the `ContentCatalog` table to retrieve a specific item by its `Id` (for example, `Id = 202`) using a projection expression to return only selected attributes such as `Title`, `ContentType`, and `Services`, and to show the consumed read capacity. This query will fetch just that one matching item from DynamoDB, return only the requested fields instead of the entire record, and report how many read capacity units the operation used.

Query options I could add to my query are things like `ProjectionExpression`, which lets me choose only the specific attributes I want returned; `ConsistentRead`, which ensures I get the most up-to-date version of the item instead of an eventually consistent read; and `ReturnConsumedCapacity`, which reports how many read capacity units were used by the request. These options help control the amount of data returned, the accuracy of the read, and the visibility into how much capacity the operation consumes.



```
-- $ aws dynamodb get-item --table-name ContentCatalog --key '{"Id":{"N":"101"}}' --consistent-read --projection-expression "Title, ContentType, Services" --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 1.0
  }
}
-- $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"202"}}' \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "Item": {
    "ContentType": {
      "S": "Video"
    },
    "Title": {
      "S": "Don't miss out!"
    }
  },
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 0.5
  }
}
-- $
```



Transactions

A transaction is an operation in DynamoDB that allows multiple actions such as writes or updates to be performed together so that they either all succeed or all fail, ensuring atomicity and consistency across items and tables.

I ran a transaction using the `aws dynamodb transact-write-items` command, which allowed me to perform multiple operations atomically in a single request. This transaction did two things: it added a new comment to the Comment table with the comment text, timestamp, and user, and it updated the matching forum topic in the Forum table by incrementing its Comments count by one.

```
~ $ aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[
> {
>   "Put": {
>     "TableName" : "Comment",
>     "Item" : {
>       "Id" : {"S": "Events/Do a Project Together - NextWork Study Session"},
>       "CommentDateTime" : {"S": "2024-9-27T17:47:30Z"},
>       "Comment" : {"S": "Excited to attend!"},
>       "PostedBy" : {"S": "User Connor"}
>     }
>   },
>   {
>     "Update": {
>       "TableName" : "Forum",
>       "Key" : {"Name" : {"S": "Events"}},
>       "UpdateExpression": "ADD Comments :inc",
>       "ExpressionAttributeValues" : { ":inc": {"N" : "1"} }
>     }
>   }
> ]'
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

